

ByteFloats.jl benchmark report

Generated: 2026-07-18 08:44 UTC · Julia 1.12.6 · cascadelake (1 threads) · Float128
paths: enabled · Chairmarks 1.3.1

Reference format for per-operation tables: `Binary8p4se` under `(NearestTiesToEven, SatNone)`. Every table names its operand class: the scalar-operation tables appear in four variants — all code points (NaN and $\pm\text{Inf}$ sampled), NaN excluded, finite-only, and per-operation in-domain — and every other sampled table uses the all-code-points pool, identified in its note. Times are per call; medians with minima alongside. Methodology per the recorded benchmark doctrine: type-parameterized barriers, untimed setup, specialization preflight.

Core primitives

The decode/compare/step/classify layer plus the projection engine. Operands: all code points — NaN and $\pm\text{Inf}$ sampled.

operation	median	min	allocs
decode	2.7 ns	2.7 ns	0
order_key	3.2 ns	3.2 ns	0
$x < y$	3.5 ns	3.5 ns	0
TotalOrder	4.3 ns	3.8 ns	0
Class	3.8 ns	2.7 ns	0
NextGreaterThan	3.8 ns	2.9 ns	0
project (RNE·SatNone)	23.0 ns	3.3 ns	0
project (StochasticA[8], R drawn)	23.0 ns	3.6 ns	0

Scalar operations — unary (30) — safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (`Sqrt`, `RSqrt`, `Log`, `Log2`, `LogOnePlus`, `Recip`, `Divide`, `ArcSin`, `ArcCos`, `ArcCosh`, `ArcTanh`) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	median	min	allocs
-----------	--------	-----	--------

Negate	22.7 ns	12.8 ns	0
Abs	24.5 ns	13.3 ns	0
Sqrt	50.3 ns	14.9 ns	0
RSqrt	53.2 ns	24.7 ns	0
Recip	53.4 ns	24.2 ns	0
Exp	56.2 ns	22.4 ns	0
ArcSin	58.1 ns	15.9 ns	0
Exp2	59.0 ns	26.4 ns	0
Tanh	59.5 ns	15.8 ns	0
Cosh	59.7 ns	23.9 ns	0
ExpMinusOne	60.3 ns	15.3 ns	0
ArcCos	60.9 ns	14.8 ns	0
Cos	61.9 ns	24.6 ns	0
ArcSinPi	62.0 ns	14.6 ns	0
Sinh	63.3 ns	17.3 ns	0
Log	65.1 ns	16.0 ns	0
Log2	66.0 ns	16.6 ns	0
LogOnePlus	66.3 ns	15.3 ns	0
ArcCosPi	66.7 ns	15.4 ns	0
SinPi	68.6 ns	14.4 ns	0
CosPi	69.2 ns	18.3 ns	0
ArcTan	69.3 ns	15.5 ns	0
Tan	72.1 ns	14.7 ns	0
ArcTanh	74.0 ns	14.8 ns	0
ArcTanPi	75.8 ns	16.9 ns	0
TanPi	75.8 ns	16.0 ns	0
ArcCosh	78.3 ns	13.7 ns	0
ArcSinh	80.8 ns	15.7 ns	0
Softplus	80.8 ns	38.2 ns	0

Sin	92.5 ns	15.5 ns	0
-----	---------	---------	---

Scalar operations — unary (30) — no NaN, Inf args

Operands exclude NaN and $\pm\text{Inf}$; finite datums only (zeros and subnormals kept).
Domain-restricted ops still take NaN fast rows on out-of-domain finite operands.
Sorted by median.

operation	median	min	allocs
ArcCosh	3.5 ns	3.5 ns	0
ArcCosPi	6.5 ns	3.2 ns	0
ArcCos	7.8 ns	3.8 ns	0
ArcSin	8.2 ns	3.9 ns	0
Log2	12.5 ns	3.9 ns	0
ArcTanh	14.5 ns	3.5 ns	0
Log	16.4 ns	3.9 ns	0
Abs	21.5 ns	12.8 ns	0
Negate	22.4 ns	13.8 ns	0
Sqrt	23.9 ns	3.2 ns	0
RSqrt	25.2 ns	3.0 ns	0
TanPi	44.1 ns	14.8 ns	0
Recip	51.3 ns	3.3 ns	0
ArcSinPi	53.3 ns	3.6 ns	0
Exp	55.2 ns	24.9 ns	0
Exp2	57.8 ns	22.0 ns	0
Tanh	58.5 ns	13.8 ns	0
Cosh	59.5 ns	22.3 ns	0
Sin	60.4 ns	14.4 ns	0
ExpMinusOne	60.7 ns	15.8 ns	0
Cos	61.0 ns	24.4 ns	0
Sinh	62.9 ns	16.3 ns	0
CosPi	67.0 ns	17.5 ns	0

LogOnePlus	67.1 ns	3.5 ns	0
SinPi	67.9 ns	15.5 ns	0
ArcTan	69.2 ns	14.6 ns	0
Tan	71.4 ns	13.4 ns	0
ArcTanPi	75.4 ns	14.6 ns	0
Softplus	79.4 ns	37.1 ns	0
ArcSinh	80.6 ns	15.5 ns	0

Scalar operations — unary (30) — no NaN args

Operands exclude the NaN code point; $\pm\text{Inf}$ and every finite datum are sampled.
Sorted by median.

operation	median	min	allocs
ArcCosh	3.5 ns	3.5 ns	0
RSqrt	4.8 ns	3.0 ns	0
Sqrt	5.6 ns	3.2 ns	0
Log	7.6 ns	3.3 ns	0
Abs	21.5 ns	12.4 ns	0
Negate	22.1 ns	13.0 ns	0
Log2	23.8 ns	3.6 ns	0
ArcSinPi	24.7 ns	3.5 ns	0
Recip	51.4 ns	3.4 ns	0
ArcCos	52.2 ns	3.8 ns	0
Exp	55.3 ns	13.5 ns	0
ArcSin	56.7 ns	3.8 ns	0
Tanh	57.8 ns	13.9 ns	0
Exp2	57.9 ns	12.4 ns	0
Sin	58.9 ns	3.3 ns	0
Cosh	59.5 ns	14.6 ns	0
ExpMinusOne	60.8 ns	14.4 ns	0

Cos	61.4 ns	3.6 ns	0
Sinh	62.7 ns	13.7 ns	0
ArcCosPi	64.0 ns	3.2 ns	0
ArcTanh	65.1 ns	3.6 ns	0
LogOnePlus	66.1 ns	3.6 ns	0
CosPi	67.5 ns	3.9 ns	0
SinPi	67.8 ns	3.6 ns	0
ArcTan	68.9 ns	14.3 ns	0
Tan	71.3 ns	3.7 ns	0
TanPi	74.9 ns	4.3 ns	0
ArcTanPi	75.6 ns	15.9 ns	0
ArcSinh	80.3 ns	13.6 ns	0
Softplus	80.3 ns	13.9 ns	0

Scalar operations — unary (30)

Operands drawn uniformly over ALL code points — NaN and $\pm\text{Inf}$ are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median. Transcendental rows mix special-row fast returns with enclosure-path evaluations, so these are *scalar-path* costs; bulk unary work routes through 256-byte tables (see Array kernels).

operation	median	min	allocs
ArcCosh	3.5 ns	3.5 ns	0
ArcTanh	5.6 ns	3.6 ns	0
ArcCosPi	5.9 ns	3.3 ns	0
ArcSin	6.7 ns	3.7 ns	0
Log2	7.2 ns	3.5 ns	0
Log	8.8 ns	3.3 ns	0
ArcSinPi	10.9 ns	3.6 ns	0
Sqrt	13.9 ns	3.3 ns	0
Abs	21.5 ns	3.9 ns	0

Negate	22.4 ns	3.9 ns	0
RSqrt	43.4 ns	2.9 ns	0
Recip	51.3 ns	3.3 ns	0
Exp	55.3 ns	3.6 ns	0
Exp2	57.7 ns	3.9 ns	0
Cosh	59.4 ns	3.6 ns	0
Tanh	59.7 ns	3.9 ns	0
Sin	60.2 ns	3.3 ns	0
ArcCos	60.3 ns	3.8 ns	0
Cos	60.8 ns	3.7 ns	0
Sinh	62.8 ns	3.4 ns	0
ExpMinusOne	62.9 ns	3.9 ns	0
LogOnePlus	66.9 ns	3.6 ns	0
CosPi	67.3 ns	3.3 ns	0
SinPi	67.7 ns	3.0 ns	0
ArcTan	69.2 ns	4.2 ns	0
Tan	71.2 ns	3.6 ns	0
TanPi	74.8 ns	4.0 ns	0
ArcTanPi	75.5 ns	3.9 ns	0
Softplus	79.6 ns	3.7 ns	0
ArcSinh	80.2 ns	4.0 ns	0

Scalar operations — binary (18) — safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (Sqrt, RSqrt, Log, Log2, LogOnePlus, Recip, Divide, ArcSin, ArcCos, ArcCosh, ArcTanh) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	median	min	allocs
Minimum	20.4 ns	13.0 ns	0

CopySign	21.5 ns	12.8 ns	0
MinimumMagnitudeNumber	21.5 ns	12.7 ns	0
Multiply	21.8 ns	13.9 ns	0
MaximumMagnitude	22.8 ns	21.8 ns	0
MinimumNumber	23.0 ns	15.1 ns	0
MaximumMagnitudeNumber	23.0 ns	22.4 ns	0
MinimumFinite	23.3 ns	14.0 ns	0
Maximum	23.3 ns	14.8 ns	0
MaximumFinite	23.6 ns	13.8 ns	0
MaximumNumber	24.5 ns	15.1 ns	0
MinimumMagnitude	24.7 ns	14.1 ns	0
Add	25.4 ns	17.4 ns	0
Subtract	31.6 ns	21.9 ns	0
Divide	47.5 ns	16.2 ns	0
Hypot	68.5 ns	26.2 ns	0
ArcTan2	77.1 ns	20.9 ns	0
ArcTan2Pi	82.2 ns	17.7 ns	0

Scalar operations — binary (18) — no NaN, Inf args

Operands exclude NaN and $\pm\text{Inf}$; finite datums only (zeros and subnormals kept).
Domain-restricted ops still take NaN fast rows on out-of-domain finite operands.
Sorted by median.

operation	median	min	allocs
MinimumNumber	20.2 ns	12.8 ns	0
CopySign	21.3 ns	12.8 ns	0
Maximum	22.7 ns	14.9 ns	0
Minimum	22.7 ns	14.3 ns	0
MinimumMagnitudeNumber	22.7 ns	14.2 ns	0
MinimumMagnitude	23.6 ns	13.6 ns	0
MinimumFinite	23.9 ns	13.3 ns	0

MaximumMagnitude	23.9 ns	22.1 ns	0
MaximumFinite	24.5 ns	13.4 ns	0
Add	24.8 ns	16.7 ns	0
Multiply	25.0 ns	14.2 ns	0
MaximumMagnitudeNumber	25.1 ns	24.1 ns	0
Subtract	31.5 ns	21.4 ns	0
Divide	47.6 ns	4.8 ns	0
MaximumNumber	48.9 ns	24.2 ns	0
Hypot	69.0 ns	23.8 ns	0
ArcTan2	77.7 ns	21.2 ns	0
ArcTan2Pi	82.3 ns	17.5 ns	0

Scalar operations — binary (18) — no NaN args

Operands exclude the NaN code point; $\pm\text{Inf}$ and every finite datum are sampled.
Sorted by median.

operation	median	min	allocs
MinimumNumber	20.2 ns	12.7 ns	0
CopySign	21.5 ns	12.8 ns	0
Maximum	22.5 ns	14.5 ns	0
Minimum	22.7 ns	14.5 ns	0
MinimumMagnitudeNumber	22.7 ns	14.2 ns	0
MinimumMagnitude	23.6 ns	13.7 ns	0
MaximumMagnitude	23.9 ns	14.2 ns	0
MinimumFinite	23.9 ns	13.5 ns	0
MaximumFinite	24.2 ns	13.3 ns	0
MaximumNumber	24.8 ns	14.2 ns	0
Add	24.8 ns	5.4 ns	0
MaximumMagnitudeNumber	25.1 ns	13.6 ns	0
Multiply	25.1 ns	14.3 ns	0

Subtract	31.3 ns	19.1 ns	0
Divide	47.6 ns	4.8 ns	0
ArcTan2	76.4 ns	18.9 ns	0
ArcTan2Pi	82.2 ns	17.2 ns	0
Hypot	140.5 ns	140.5 ns	0

Scalar operations — binary (18)

Operands drawn uniformly over ALL code points — NaN and $\pm\text{Inf}$ are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median.

operation	median	min	allocs
MinimumNumber	20.2 ns	12.8 ns	0
CopySign	21.5 ns	3.6 ns	0
Minimum	22.7 ns	3.6 ns	0
MinimumMagnitudeNumber	22.7 ns	14.2 ns	0
MinimumMagnitude	23.6 ns	3.6 ns	0
MinimumFinite	23.6 ns	13.7 ns	0
MaximumMagnitude	23.7 ns	3.6 ns	0
MaximumFinite	24.3 ns	13.4 ns	0
MaximumNumber	24.8 ns	14.2 ns	0
Add	24.8 ns	4.7 ns	0
MaximumMagnitudeNumber	25.3 ns	13.6 ns	0
Multiply	25.6 ns	3.6 ns	0
Maximum	26.7 ns	3.6 ns	0
Subtract	31.3 ns	8.9 ns	0
Divide	47.8 ns	4.5 ns	0
Hypot	68.8 ns	5.4 ns	0
ArcTan2	77.9 ns	8.3 ns	0
ArcTan2Pi	83.1 ns	7.9 ns	0

Scalar operations — ternary (3) — safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (Sqrt, RSqrt, Log, Log2, LogOnePlus, Recip, Divide, ArcSin, ArcCos, ArcCosh, ArcTanh) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	median	min	allocs
Clamp	24.7 ns	14.3 ns	0
FMA	28.6 ns	22.9 ns	0
FAA	30.4 ns	21.7 ns	0

Scalar operations — ternary (3) — no NaN, Inf args

Operands exclude NaN and $\pm\text{Inf}$; finite datums only (zeros and subnormals kept). Domain-restricted ops still take NaN fast rows on out-of-domain finite operands. Sorted by median.

operation	median	min	allocs
Clamp	24.2 ns	15.4 ns	0
FAA	30.7 ns	24.2 ns	0
FMA	31.9 ns	24.0 ns	0

Scalar operations — ternary (3) — no NaN args

Operands exclude the NaN code point; $\pm\text{Inf}$ and every finite datum are sampled. Sorted by median.

operation	median	min	allocs
Clamp	24.2 ns	15.4 ns	0
FAA	30.7 ns	20.5 ns	0
FMA	31.8 ns	19.0 ns	0

Scalar operations — ternary (3)

Operands drawn uniformly over ALL code points — NaN and $\pm\text{Inf}$ are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median.

operation	median	min	allocs
-----------	--------	-----	--------

Clamp	24.2 ns	3.6 ns	0
FAA	30.7 ns	8.0 ns	0
FMA	31.8 ns	8.5 ns	0

Format sensitivity

Same three binary ops across formats; Binary8p1uf exercises the wide-exponent-spread escalations, small-K formats the tiny-table regime. Operands: all code points — NaN and $\pm\text{Inf}$ sampled.

operation	median	min	allocs
Add(Binary8p4se)	24.8 ns	4.8 ns	0
Divide(Binary8p4se)	47.6 ns	4.5 ns	0
Multiply(Binary8p4se)	25.0 ns	3.6 ns	0
Add(Binary8p3sf)	22.7 ns	4.7 ns	0
Divide(Binary8p3sf)	43.9 ns	4.8 ns	0
Multiply(Binary8p3sf)	21.8 ns	3.3 ns	0
Add(Binary8p1uf)	178.9 ns	7.7 ns	0
Divide(Binary8p1uf)	23.8 ns	4.5 ns	0
Multiply(Binary8p1uf)	21.2 ns	3.6 ns	0
Add(Binary5p2se)	27.1 ns	4.5 ns	0
Divide(Binary5p2se)	25.7 ns	4.4 ns	0
Multiply(Binary5p2se)	24.5 ns	3.6 ns	0
Add(Binary3p1se)	16.7 ns	4.4 ns	0
Divide(Binary3p1se)	15.6 ns	4.4 ns	0
Multiply(Binary3p1se)	14.7 ns	3.8 ns	0

Projection by rounding/saturation mode

project(Binary8p4se, p, x) over the mode vocabulary (stochastic budgets N = 8). Operands: all code points — NaN and $\pm\text{Inf}$ sampled.

operation	median	min	allocs
NearestTiesToEven	19.8 ns	3.6 ns	0

NearestTiesToAway	19.3 ns	5.0 ns	0
TowardPositive	19.2 ns	5.6 ns	0
TowardNegative	19.4 ns	5.6 ns	0
TowardZero	18.0 ns	5.3 ns	0
ToOdd	22.7 ns	3.6 ns	0
StochasticA[8]	23.3 ns	3.9 ns	0
StochasticB[8]	19.5 ns	3.9 ns	0
StochasticC[8]	19.5 ns	3.6 ns	0
RNE · SatFinite	20.3 ns	3.9 ns	0
RNE · SatPropagate	21.5 ns	3.6 ns	0

Array kernels (vmap)

Warm caches: table specializations prebuilt, so table rows measure the gather; scalar-loop rows measure the full compute pipeline per element. Operands: all code points — NaN and $\pm\text{Inf}$ sampled.

operation	median	min	allocs	per element
vmap unary (table gather), n=65536	36.1 μs	33.24 μs	0	0.55 ns/elem — 1.82 Gelem/s
vmap binary (table gather), n=65536	47.5 μs	44.45 μs	0	0.72 ns/elem — 1.38 Gelem/s
vmap ternary (scalar loop), n=65536	2.28 ms	2.22 ms	0	34.77 ns/elem — 0.03 Gelem/s
vmap binary stochastic (scalar loop), n=65536	1.8 ms	1.74 ms	0	27.52 ns/elem — 0.04 Gelem/s
vmap unary through PackedVector, n=65536	243.29 μs	227.31 μs	519	3.71 ns/elem — 0.27 Gelem/s

Sorting (64 K values)

Counting sort is installed as the default algorithm for Binary vectors. Operands: all code points — NaN and $\pm\text{Inf}$ sampled.

operation	median	min	allocs
-----------	--------	-----	--------

sort! (counting sort via defalg), n=65536	336.45 μs	324.72 μs	2
sort! (stock comparison sort), n=65536	4.36 ms	4.31 ms	3
sort! rev=true (counting sort), n=65536	343.5 μs	321.6 μs	2

Table builds (oracle + projection, Float128-first)

Cold cache per sample (empty_tables! in untimed setup); JIT pre-warmed. The warm-hit column is the steady-state cost of get_table when the specialization is already cached (median / min). Table entries enumerate every code point by construction (NaN and ±Inf included).

operation	median	min	allocs	warm hit
Exp(8p4se) (256 entries)	86.41 μs	84.14 μs	257	179.4 ns / 176.8 ns
Tanh(8p4se) (256 entries)	96.05 μs	93.09 μs	257	179.9 ns / 176.8 ns
Add(8p4se×8p4se) (64 K entries)	70.44 ms	65.94 ms	785668	182.4 ns / 178.7 ns
Divide(8p4se×8p4se) (64 K)	71.08 ms	66.33 ms	784906	182.9 ns / 178.2 ns
Add(8p1uf×8p1uf) (64 K, wide-spread)	118.48 ms	108.18 ms	1587030	181.3 ns / 176.8 ns

Block and scaled operations

Elements Binary8p4se, scales Binary8p1uf, B = 32. Operands: all code points — NaN and ±Inf sampled.

operation	median	min	allocs	per lane
BlockAdd (B=32)	2.29 μs	2.13 μs	35	71.6 ns/lane
BlockDotProduct → 8p4se (B=32)	2.94 μs	391.2 ns	66	92.0 ns/lane
BlockReduceAdd → 8p4se (B=32)	992.7 ns	58.4 ns	0	31.02 ns/lane

ConvertToBlockMaxAbsFinite (B=32)	4.45 μ s	4.19 μ s	75	138.99 ns/lane
--------------------------------------	--------------	--------------	----	-------------------

Conversions and packed storage

Operands: all code points — NaN and $\pm$ Inf sampled.

operation	median	min	allocs	per element
T(::UInt8) code-point constructor	2.9 ns	2.9 ns	0	
rawvalue (unchecked kernel route)	2.9 ns	2.9 ns	0	
T(::Float64) numeric constructor (projects)	22.4 ns	3.6 ns	0	
Convert 8p4se $\rightarrow$ 8p3se (scalar)	22.7 ns	3.7 ns	0	
Float64 $\rightarrow$ 8p4se (project)	22.4 ns	3.6 ns	0	
PackedVector pack, n=65536	85.13 μ s	76.68 μ s	3	1.3 ns/elem
PackedVector unpack (collect), n=65536	59.28 μ s	58.22 μ s	3	0.9 ns/elem

*All numbers from this machine/run; absolute values vary by host. Regenerate with
julia --project=benchmark benchmark/benchmarking.jl.*